

Автоматизация деплоя, миграции, интеграции для self-hosted таблиц

Презентация по итогам итерации №3

Заказчик: Заславский М.М.

Исполнители: Попандопуло А.Г. гр. 1304

Марков М.М., Сычев Н.С., Гладилин Е.О., Журавлев Д.В., Яковлев Д.С. гр. 1381

План текущей итерации

Формы: доработать сценарий сбора данных, реализовать динамическую привязку форм к таблицам для каждого пользователя.

Адаптер импорта: реализовать модуль загрузки данных из внешних источников в систему.

Резервное копирование: реализовать команды создания и восстановления резервных копий БД и пользовательских файлов.

Статус: реализовать скрипт проверки состояния всех компонентов системы (контейнеры, доступность API, соединение с БД).

Мониторинг ресурсов: реализовать сбор и отображение метрик нагрузки (CPU, RAM, активные сессии, время отклика).

Портирование таблиц: начать миграцию имеющихся таблиц на нашу систему.

Результаты текущей итерации

Формы и таблицы: связка переведена на нативный механизм Nextcloud Forms с периодической синхронизацией через cron; внешний workflow-движок больше не требуется.

Адаптер импорта: загрузка CSV-выгрузок (GitLogger, Stepik/Moodle и др. CSV) в существующую таблицу по ключу - обновление совпавших строк и добавление новых.

Резервное копирование: команды создания, просмотра и восстановления резервных копий с разделением на состояние системы и пользовательские файлы.

Проверка состояния системы: статус контейнеров, соединение с БД, доступность API; режим ожидания готовности.

Мониторинг ресурсов: метрики для CPU/RAM по контейнерам, время отклика API, активные сессии, свободное место.

Контейнеризация CLI: wrapper bin/table-cli поднимает manage.py внутри Docker-образа, Python и зависимости на хосте больше не требуются.

План на следующую итерацию

- Нагрузочное тестирование: реализовать сценарий эмуляции типовой нагрузки с метриками отклика и потребления ресурсов.
- Подготовить production-ready конфигурацию.
- Integration-тесты и настройка GitHub Actions.
- Согласование и финальные доработки по обратной связи со стороны заказчика.

Инструкции по запуску

README

mse1n2026-table

Self-hosted онлайн таблицы для кафедры на базе Nextcloud + OnlyOffice. Стек поднимается одной командой через Docker Compose, эксплуатация (пользователи, загрузка таблиц, бэкапы, мониторинг, импорт из внешних CSV) - через CLI `manage.py` / `bin/table-cli`.

Требования

- [Docker](#) и [Docker Compose](#)
- Git
- Python 3.x - только если запускаете CLI напрямую через `python3 manage.py`. Если идете через `./bin/table-cli` (см. ниже), Python на хосте не требуется.

Поддерживаемые ОС

- Linux / macOS - основной режим, все работает напрямую.
- Windows - рекомендуется [WSL2](#) с Docker Desktop. При работе из WSL действия аналогичны Linux. Без WSL - нужен Docker Desktop, hosts-файл лежит по `C:\Windows\System32\drivers\etc\hosts`

Подготовка хоста

В hosts-файл добавить запись (один раз):

```
127.0.0.1 nextcloud.localhost
```

На Linux / macOS / WSL:

```
echo "127.0.0.1 nextcloud.localhost" | sudo tee -a /etc/hosts
```

На Windows (PowerShell от администратора):

```
Add-Content -Path "$env:windir\System32\drivers\etc\hosts" -Value "`n127.0.0.1`nnextcloud.local
```

README

Дефолтный URL стека после старта - <http://nextcloud.localhost:8080>. Порт меняется через `NEXTCLOUD_PORT` в `deploy/.env`, `hostname` - через `NEXTCLOUD_HOSTNAME` (с синхронной правкой hosts-файла).

Авторизация в Nextcloud

После `deploy up` веб-интерфейс открывается по <http://nextcloud.localhost:8080> со следующими дефолтными admin-кредами:

- **Логин:** `admin`
- **Пароль:** `super_secure_password`

Эти значения лежат в `deploy/.env` под именами `NEXTCLOUD_ADMIN_USER` и `NEXTCLOUD_ADMIN_PASSWORD`. Перед production-развертыванием - поменять и пересоздать стек (`deploy down` -> удалить volume `app_data` -> `deploy up`), иначе старый пароль в БД переживет правку `.env`.

Запуск CLI: два равнозначных варианта

А. Контейнеризованный CLI

Из корня репозитория:

```
./bin/table-cli <subcommand> [args...]
```

Wrapper собирает образ `mse1h2026-table-cli` при первом вызове и проксирует аргументы в `manage.py` внутри контейнера. На хосте нужен только Docker.

Все примеры ниже с `python3 manage.py` имеют точный эквивалент `./bin/table-cli` с теми же флагами:

```
./bin/table-cli deploy up
./bin/table-cli deploy demo
./bin/table-cli users list --details
./bin/table-cli backup create --components data --name nightly
```

Как это работает. Контейнер cli монтирует `/var/run/docker.sock` - это нужно, чтобы `backup` / `restore` / `monitor` / `deploy` * могли управлять остальными сервисами стека (`docker compose exec` для `occ`,

Формы -> таблицы через cron-worker

Видео... Файлы Фото Событ... **Формы**

+ Новая форма

Ваши формы

Сбор сведений

Submits 38

Проверка чисел

Первая форма 6

Ссылка на GitHub

Описание (поддерживается форматирование с использованием Markdown)

Можно написать короткий ответ

Telegram для связи

Через собаку @

Можно написать короткий ответ

Дата рождения

Описание (поддерживается форматирование с использованием Markdown)

Можно выбрать дату

Почта

Описание (поддерживается форматирование с использованием Markdown)

Пользователи могут ввести адрес электронной почты

+ Добавить вопрос

Сбор сведений

0 ответов

Всего Ответы

Создать электронную таблицу

Настройка формы

Параметры публикации Настройки

Select account

Поиск пользователя, группы или команды...

- Поделиться ссылкой
- Внутренняя ссылка
- Разрешить доступ для всех зарегистрированных учетных записей